

Research Note: Why LLD Practice Is Hard to Learn (and to Evaluate)

1. The Learner Problem

Low-Level Design (LLD) — designing classes, responsibilities, interfaces, and their interactions for a bounded problem (e.g., “design a parking lot,” “design an elevator system”) — is a core skill for mid-to-senior software engineering interviews and day-to-day system design work. Yet it is one of the hardest skills to practice alone, for reasons that are structural, not just a matter of motivation:

1.1 No objective test case exists. Unlike an algorithm problem, an LLD submission has no single correct answer that a test harness can check. There is no equivalent of “does this pass the hidden test cases” for a design — the artifact being judged is a set of classes and relationships, not a function’s output.

1.2 Multiple valid architectural solutions coexist. Two designs can both “work” for the stated requirements while differing sharply in extensibility, coupling, and adherence to SOLID principles. A `Vehicle` class with an if/else on type and a polymorphic `Vehicle` hierarchy both compile and both satisfy the parking-lot requirements today — the difference only shows up when a new vehicle type is added later. Judging which is “better” requires reasoning about futures the learner didn’t write, not just the code in front of you.

1.3 The trade-offs are subjective and context-dependent. Whether a given coupling is “acceptable” depends on expected scale, team size, and how likely a requirement is to change — judgment calls that differ between reviewers and can’t be reduced to a checklist.

1.4 Senior-engineer feedback is the bottleneck. Real feedback on trade-offs usually only comes from a senior engineer or an experienced interviewer — a resource most learners don’t have on demand, and one that doesn’t scale: mock interviews and mentor time are expensive and inconsistent session to session.

Without a tight feedback loop (attempt → specific critique → understand the gap → retry), learners either repeat the same anti-patterns indefinitely or drift into passive content consumption (reading, watching) instead of active practice.

2. Existing Solutions Evaluated

LeetCode / HackerRank. Excellent for data structures, algorithms, and syntax — optimized for correctness against hidden test cases. Structurally unable to help with object decomposition or responsibility design: there is no test case that captures “is this coupling acceptable?” A handful of platforms bolt on “system design” question banks, but grading is either absent (read-only reference answers) or shallow (multiple choice).

Static LMS / video courses. Good for building theoretical knowledge of SOLID, GoF patterns, and case-study walkthroughs. Zero interactive feedback: the content is one-directional, and there is no mechanism for a learner to submit their own design and receive feedback on their trade-offs specifically. A learner can watch ten videos on the Strategy pattern and still not recognize where their own code needs it.

Unconstrained LLM chat (ChatGPT, Claude, etc.). The most capable of the three at generating relevant critique, but unreliable as a practice tool in its default form:

- **Sycophancy.** A general-purpose chat assistant tends toward “This looks great! A few minor suggestions...” even for designs with real structural problems, because it isn’t anchored to a specific standard of judgment.
- **No rubric grounding.** Without a fixed set of dimensions to score against, the same submission can get very different critiques on different days, making it hard for a learner to track whether they’re actually improving.
- **Non-reproducible advice.** Feedback isn’t structured (no consistent criterion → score → evidence shape), so it can’t be compared across attempts or aggregated into a sense of progress.

The gap: there is no lightweight, always-available tool that takes a free-form LLD submission and returns structured, rubric-anchored, actionable feedback — consistently enough to support a tight practice loop and to make progress visible.

3. Product Direction

The system built here is a focused practice loop: **Choose problem → design in text → submit → get structured feedback → review → retry.** Two design choices directly answer the failure modes above:

- **A fixed, structured rubric** — four architectural dimensions (Responsibility Clarity, Coupling & Cohesion, SOLID & Pattern Adherence, Extensibility & Trade-offs) that every submission is scored against, each with an evidence quote, a concern, and a concrete suggestion. This directly counters chat-LLM sycophancy and inconsistency: the model is instructed to ground every score in the submission text itself, using explicit scoring bands (90–100 exceptional, 70–89 good with minor leakage, 50–69 noticeable anti-patterns, below 50 fundamental misunderstanding).
- **Deterministic pre-checks before AI judgment.** Trivial submissions (empty, or under a minimum length) are rejected instantly at the API boundary, before any LLM call — so the AI’s judgment is reserved for submissions that are actually worth judging, and a learner never sees “is your text present” mixed in with real architectural critique.

I built this in about 5 hours, so the scope stays intentionally narrow: text-based submissions only (no diagram parsing yet), three seeded practice problems (Parking Lot, Elevator System, Vending Machine), and a single LLM-backed evaluator. What I didn’t cut corners on was the domain model — it’s built so that broader submission formats and additional evaluator types (human review, rule-based checks) drop in as extensions later, not rewrites. See the accompanying *Design Note* for how that’s wired.